

# Do sprint.md ao MVP no ar

Passo a passo e biblioteca de prompts para construir o MVP do grupo no Antigravity: TypeScript e Python num PWA, versionado no GitHub e publicado automaticamente no GitHub Pages.

Curso de Capacitação SESP/MT · Professor Renato Rosa · Dia 8

---

# Sumário

## ABERTURA

|   |                                                                   |   |
|---|-------------------------------------------------------------------|---|
| 1 | Onde estamos e o que vamos construir hoje                         | 3 |
| 2 | Como usar este caderno                                            | 4 |
| 3 | Antes de começar: vocês rodaram estes dois prompts no NotebookLM? | 4 |
| 4 | Os prompts do NotebookLM, versão técnica                          | 5 |

## PARTE 1 · O QUE DÁ PARA PUBLICAR

|   |                                              |   |
|---|----------------------------------------------|---|
| 5 | A verdade sobre "full stack" no GitHub Pages | 8 |
| 6 | A arquitetura do MVP: quem faz o quê         | 9 |
| 7 | A estrutura de pastas do repositório         | 9 |

## PARTE 2 · TRABALHANDO COM O AGENTE

|    |                                                              |    |
|----|--------------------------------------------------------------|----|
| 8  | Anatomia de um bom prompt de desenvolvimento                 | 11 |
| 9  | O ciclo de cada tarefa: planejar → gerar → testar → commitar | 12 |
| 10 | Regras de segurança ao trabalhar com o agente                | 13 |

## PARTE 3 · SPRINT 0 — PREPARAR O TERRENO

|    |                                                                   |    |
|----|-------------------------------------------------------------------|----|
| 11 | Passo 1 — Instalar Node.js e Python (e conferir o Git)            | 14 |
| 12 | Passo 2 — Criar o projeto e trazer o contexto.md e o sprint.md    | 15 |
| 13 | Passo 3 — Versionar e publicar o repositório no GitHub            | 16 |
| 14 | Passo 4 — O primeiro prompt: o agente lê o contexto e o sprint.md | 17 |
| 15 | Passo 5 — O arquivo de regras do projeto (a "memória" do agente)  | 18 |
| 16 | Passo 6 — Configurar o Vite para o GitHub Pages (o "base")        | 18 |

## PARTE 4 · CONSTRUINDO SPRINT A SPRINT

|    |                                                                 |    |
|----|-----------------------------------------------------------------|----|
| 17 | O mapa das 5 sprints do MVP                                     | 20 |
| 18 | Sprint 1 — Modelagem e dados sintéticos com Python              | 21 |
| 19 | Sprint 2 — Regras de negócio, papéis e auditoria em TypeScript  | 23 |
| 20 | Sprint 3 — Telas, painel e navegação                            | 25 |
| 21 | Sprint 4a — Transformar em PWA (instalável e offline)           | 26 |
| 22 | Sprint 4b — Homologação: testes, LGPD e revisão                 | 28 |
| 23 | Sprint 5 — Deploy automático no GitHub Pages com GitHub Actions | 29 |
| 24 | Versionamento profissional: branches, commits, PRs e tags       | 31 |
| 25 | README e transparência: documentando o que a IA fez             | 33 |

## PARTE 5 · BIBLIOTECA DE PROMPTS

|    |                                                |    |
|----|------------------------------------------------|----|
| 26 | Prompts de planejamento e entendimento         | 34 |
| 27 | Prompts de construção por camada               | 35 |
| 28 | Prompts de depuração: quando algo dá errado    | 36 |
| 29 | Prompts de revisão, refatoração e documentação | 37 |
| 30 | Prompts ruins × prompts bons                   | 38 |

## FECHAMENTO

|    |                                                     |    |
|----|-----------------------------------------------------|----|
| 31 | Problemas comuns e como resolver                    | 40 |
| 32 | Checklist de entrega do Dia 8 (Definição de Pronto) | 41 |
| 33 | Glossário rápido                                    | 42 |
| 34 | Quiz rápido                                         | 43 |

## ABERTURA

# Abertura

O que vamos construir hoje, como usar este caderno e a conferência dos arquivos que vieram do NotebookLM.

1. Onde estamos e o que vamos construir hoje
2. Como usar este caderno
3. Antes de começar: vocês rodaram estes dois prompts no NotebookLM?
4. Os prompts do NotebookLM, versão técnica

§ 1

## Onde estamos e o que vamos construir hoje

Na Aula 7 vocês planejaram o projeto: definiram a dor, os usuários, os dados, o BPMN, a arquitetura, o uso da IA e as sprints. Depois levaram tudo do **NotebookLM** para o **Antigravity** em dois arquivos: o `contexto.md` (o resumo do projeto) e o `sprint.md` (as sprints técnicas). Hoje esses dois arquivos viram **software de verdade**: um MVP instalável no celular, versionado no GitHub e publicado na internet pelo GitHub Pages.



## O que vocês levam do Dia 8

- Um repositório no GitHub com o código do MVP, histórico de commits e README;
- o MVP publicado em `https://SEU-USUARIO.github.io/NOME-DO-REP0/`, instalável como aplicativo (PWA);
- deploy automático: cada `push` na branch `main` atualiza o site sozinho;
- uma biblioteca de prompts para continuar desenvolvendo nas próximas aulas.

### EXEMPLO USADO NESTE CADERNO

Para os exemplos ficarem concretos, usamos o **Portal de Pedidos de Compra**, o To-Be do exemplo da Aula 7 (Solicitante abre o pedido → Sistema valida → Cotação → Controladoria audita → Secretário aprova no painel). O repositório de exemplo se chama `mvp-pedidos-compra`. Troquem sempre pelo projeto de vocês.

## Como usar este caderno

O caderno é um **roteiro para seguir em ordem**: cada Parte termina com algo funcionando e um commit no GitHub. Estes são os elementos que se repetem:



### Bloco de prompt

Texto para copiar e colar no **painel Agent** do Antigravity (ou no chat do NotebookLM, quando indicado). No HTML há um botão **Copiar** em cada bloco.



### [CAMPOS EM AMARELO]

Dentro dos prompts, o que está entre colchetes é para **substituir** pelos dados do projeto de vocês antes de enviar.



### Bloco de terminal

Comandos para digitar no terminal do Antigravity (**Terminal** → **New Terminal**). Uma linha por vez; o que vem depois de `#` é comentário.



### Bloco de arquivo

Conteúdo de um arquivo do projeto (o nome aparece no canto do bloco). Serve de gabarito para conferir o que o agente gerou.



### Caixa vermelha

Armadilha ou regra que não pode ser quebrada (LGPD, comandos que apagam arquivos, chaves secretas).



### Caixa verde

Dica, boa prática ou a mensagem de commit sugerida para aquela etapa.

| Parte                        | O que vocês fazem                                     | Termina com                                         |
|------------------------------|-------------------------------------------------------|-----------------------------------------------------|
| 1 · O que dá para publicar   | Entender o que "full stack" vira num site estático    | A arquitetura e a estrutura de pastas definidas     |
| 2 · Trabalhando com o agente | Aprender o formato de prompt e o ciclo de cada tarefa | Regras de segurança combinadas no grupo             |
| 3 · Sprint 0                 | Instalar, criar o projeto e publicar o repositório    | Repositório no GitHub + agente conhecendo o projeto |
| 4 · Sprints 1 a 5            | Dados, regras, telas, PWA, testes e deploy            | MVP no ar em <code>github.io</code>                 |
| 5 · Biblioteca de prompts    | Consultar quando precisar                             | —                                                   |
| Fechamento                   | Resolver problemas e conferir a entrega               | Checklist completo                                  |

## Antes de começar: vocês rodaram estes dois prompts no NotebookLM?

Tudo o que vem depois depende da qualidade do `contexto.md` e do `sprint.md`. O agente do Antigravity **não conhece o projeto de vocês**: ele só sabe o que está escrito nesses arquivos. Confirmem primeiro que os dois existem e foram gerados com estes prompts:

| Arquivo     | Prompt usado no NotebookLM                                                            | Para que serve no AntigraVity                                                                      |
|-------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| contexto.md | "Anotar todo o contexto fornecido até aqui para a ideiação e prototipagem do projeto" | A "memória" do projeto: problema, usuários, dados, regras. O agente consulta antes de cada tarefa. |
| sprint.md   | "Gere sprints de execução do projeto de forma técnica em (JS, TS, Python em PWA)"     | O roteiro de construção: o que fazer em cada sprint. O agente executa uma tarefa por vez.          |

## Teste rápido: os arquivos de vocês estão bons o suficiente?

Abram os dois arquivos no AntigraVity e marquem o que eles contêm:

- ☐ **contexto.md** descreve a dor com número (ex.: "10 dias para consolidar")
- ☐ **contexto.md** lista os papéis de usuário (Admin, Editor, Leitor, Aprovador) e o que cada um faz
- ☐ **contexto.md** lista as entidades de dados com seus campos (ex.: Pedido: protocolo, unidade, valor...)
- ☐ **contexto.md** diz quais campos são sensíveis (LGPD) e como mascarar
- ☐ **contexto.md** descreve o fluxo To-Be passo a passo
- ☐ **sprint.md** tem sprints numeradas, cada uma com tarefas e um critério de "pronto"
- ☐ **sprint.md** diz em qual linguagem fica cada parte (TS, JS, Python)
- ☐ **sprint.md** considera que o site será **estático**, no GitHub Pages

### RESULTADO DO TESTE

**7 ou 8 marcados:** sigam em frente. **4 a 6:** usem os prompts melhorados da próxima seção para complementar. **3 ou menos:** gerem os arquivos de novo com os prompts melhorados antes de escrever código. Os 20 minutos que isso leva economizam horas de retrabalho com o agente.

§ 4

## Os prompts do NotebookLM, versão técnica

Os prompts originais funcionam, mas deixam o NotebookLM decidir sozinho o formato e o nível de detalhe. As versões abaixo pedem uma **estrutura fixa**, com as informações que o agente de código mais usa. Rodem no **mesmo notebook** do grupo, onde estão as fontes da Aula 7, e salvem a resposta como arquivo `.md`.

PROMPT

Atue como analista de requisitos de software do setor público. Com base em TODAS as fontes e conversas deste notebook,

escreva o documento "contexto.md" do projeto [NOME DO SISTEMA] em Markdown, com EXATAMENTE estas seções:

1. Visão geral: o problema (dor) em 1 frase com número, o setor da SESP-MT e a solução proposta.
2. Objetivos e KPIs: 2 a 4 KPIs, cada um com baseline (hoje), meta e como medir.
3. Usuários e papéis: tabela Papel | Quem é na SESP | O que pode fazer | O que NÃO pode fazer (papéis A, E, L, X).
4. Fluxo To-Be: passos numerados, indicando qual papel executa cada passo e o que o sistema faz sozinho.
5. Entidades de dados: para cada entidade, tabela Campo | Tipo (texto, número, data, lista) | Obrigatório | Exemplo.
6. Regras de negócio: lista numerada RN01, RN02... (ex.: "RN03 - pedido acima de R\$ 50 mil exige auditoria").
7. Dados sensíveis (LGPD): quais campos, classificação (pessoal/sensível) e técnica de proteção (máscara, hash, omitir).
8. Telas do MVP: nome da tela, objetivo, papel que usa, elementos principais (KPIs, gráfico, tabela, formulário).
9. Restrições técnicas: site estático no GitHub Pages, sem servidor e sem banco real, apenas dados sintéticos.
10. Fora do escopo do MVP: o que fica para depois.
11. Dúvidas em aberto: o que ainda não foi decidido pelo grupo.

Não invente informação: se algo não estiver nas fontes, escreva "A DEFINIR" no lugar.

PROMPT

Atue como tech lead de um projeto de software. Com base no contexto do projeto [NOME DO SISTEMA] neste notebook,

gere o documento "sprint.md" em Markdown com o plano técnico de construção do MVP.

Stack obrigatória:

- Front-end: TypeScript + Vite, como PWA (vite-plugin-pwa), com rotas por hash (#/tela).
- JavaScript: apenas onde o Vite/PWA gerar (service worker, arquivos de configuração).
- Python 3.12: scripts de geração de dados sintéticos (saída em public/data/\*.json), mascaramento LGPD e testes (pytest).
- Testes do front-end: Vitest.
- Hospedagem: GitHub Pages (site estático), com deploy automático por GitHub Actions a cada push na main.
- Sem servidor e sem banco de dados real: dados em JSON + armazenamento local no navegador (IndexedDB ou localStorage).

Gere 5 sprints (1 Fundação e dados, 2 Regras de negócio e papéis, 3 Telas, 4 PWA e homologação, 5 Deploy e documentação).

Para CADA sprint escreva:

- Objetivo (1 frase) e entregável demonstrável.
- Tarefas numeradas (S1.1, S1.2...), cada uma com: arquivos a criar ou alterar, linguagem, e critério de aceite testável.
- Testes da sprint (o que o Vitest ou o pytest deve verificar).
- Mensagem de commit sugerida no padrão Conventional Commits (feat:, fix:, test:, docs:, chore:).

Termine com uma "Definição de Pronto" do MVP em checklist.

#### JÁ TÊM UM SPRINT.MD QUE PEDE SERVIDOR E BANCO DE DADOS?

Muitos grupos receberam sprints com FastAPI/Flask, PostgreSQL ou login com senha. Não joguem fora: a próxima Parte mostra como isso vira um site estático, e no Passo 4 da Sprint 0 há um prompt que pede ao agente para reescrever o `sprint.md` para o GitHub Pages sem perder o que foi planejado.

#### LGPD VALE AQUI TAMBÉM

O NotebookLM é um serviço externo. Se alguma fonte do notebook tiver nome, CPF ou matrícula reais, removam antes de gerar os arquivos. Os dois `.md` vão para um repositório **público** no GitHub.

## PARTE 1

# O que dá para publicar

O GitHub Pages só entrega arquivos prontos. Aqui vocês veem como cada peça "full stack" do sprint.md vira um site estático, qual o papel de cada linguagem e como organizar as pastas.

1. A verdade sobre "full stack" no GitHub Pages
2. A arquitetura do MVP: quem faz o quê
3. A estrutura de pastas do repositório

§ 5

## A verdade sobre "full stack" no GitHub Pages

O GitHub Pages é gratuito e simples, mas tem uma regra que muda tudo: **ele só entrega arquivos prontos** (HTML, CSS, JavaScript, imagens, JSON). Ele não executa Python, não tem banco de dados e não guarda nada que o usuário digita. Cada parte "full stack" do sprint.md precisa ter um equivalente estático:

| O que o sprint.md pede               | Como fica no MVP do GitHub Pages                                                                                              | Linguagem                |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| Backend / API (FastAPI, Flask, Node) | Uma camada <code>src/services/</code> em TypeScript que lê JSON, com a mesma "cara" de uma API                                | TS                       |
| Banco de dados (PostgreSQL, MySQL)   | Arquivos <code>public/data/*.json</code> gerados por script Python, mais IndexedDB/localStorage para o que o usuário cadastra | Python + TS              |
| Geração de dados, ETL, relatórios    | Scripts em <code>scripts/*.py</code> , executados no computador e no GitHub Actions antes do build                            | Python                   |
| Login e senha                        | Seletor "Ver como: Admin / Editor / Leitor / Aprovador" (RBAC simulado, como combinado na Aula 7)                             | TS                       |
| Logs de auditoria                    | Registro local (quem, papel, ação, antes/depois, data/hora) com exportação em CSV                                             | TS                       |
| Mascaramento LGPD                    | Feito no script Python <b>antes</b> de gerar o JSON: o dado completo nunca chega ao site                                      | Python                   |
| App de celular                       | PWA: instalável pelo navegador e funcionando offline                                                                          | TS + JS (service worker) |
| Servidor de deploy                   | GitHub Actions compila e publica sozinho a cada push                                                                          | YAML                     |

### REGRA DE OURO: TUDO NO GITHUB PAGES É PÚBLICO

Qualquer pessoa pode abrir o código-fonte do site e baixar os JSON. Por isso: **nenhum dado real, nenhuma senha, nenhuma chave de API** (nem de IA) no repositório. Se o agente sugerir colocar uma chave no código, recusem.

### E O CÓDIGO DE BACKEND QUE VOCÊS PLANEJARAM?

Ele pode continuar existindo como **evolução futura**. A camada `src/services/` é escrita contra uma interface (ex.: `listarPedidos()`). Hoje ela lê JSON; amanhã, quando a SESP hospedar uma API de verdade, troca-se só a implementação, e as telas não mudam. Isso se chama **padrão adaptador**.

## A arquitetura do MVP: quem faz o quê

Cada linguagem tem um papel claro. Guardem este quadro: ele responde metade das dúvidas sobre onde colocar cada código.

### Python: a "fábrica de dados"

- Gera dados sintéticos realistas ( `scripts/gerar_dados.py` );
- mascara campos sensíveis (LGPD) antes de salvar;
- valida os dados com testes ( `pytest` );
- roda **antes** do build, nunca no navegador.

### TypeScript: o aplicativo

- Tipos das entidades ( `src/domain/tipos.ts` );
- regras de negócio RN01, RN02... ( `src/domain/regras.ts` );
- papéis e permissões, auditoria, telas;
- testado com `Vitest` .

### JavaScript: a cola do PWA

- Service worker (gerado pelo `vite-plugin-pwa` );
- arquivos de build em `dist/` ;
- o TypeScript vira JavaScript no build: o navegador só executa JS.

### GitHub: versão e publicação

- Repositório: histórico de tudo (commits);
- Actions: roda Python, testes e build a cada push;
- Pages: entrega o site em `https://usuario.github.io/repo/` .



## A estrutura de pastas do repositório

Peçam ao agente para seguir esta estrutura desde o primeiro dia. Com ela, qualquer colega (e o próprio agente) sabe onde procurar cada coisa.

```

mvp-pedidos-compra/
├─ docs/
│  ├─ contexto.md           ← vindo do NotebookLM
│  ├─ sprint.md             ← vindo do NotebookLM
│  └─ REGRAS-DO-PROJETO.md  ← regras que o agente segue sempre
├─ scripts/                 ← PYTHON
│  ├─ gerar_dados.py        ← dados sintéticos → public/data/*.json
│  └─ test_gerar_dados.py   ← pytest: LGPD e consistência
├─ public/
│  ├─ data/                 ← JSON gerados (não editar à mão)
│  ├─ icon.svg              ← ícone-fonte do PWA
│  └─ pwa-192x192.png ...   ← ícones gerados
├─ src/                     ← TYPESCRIPT
│  ├─ domain/               ← tipos.ts, regras.ts (+ regras.test.ts)
│  ├─ auth/                 ← papeis.ts (RBAC simulado)
│  ├─ services/             ← api.ts, auditoria.ts, armazenamento.ts
│  ├─ pages/                ← uma tela por arquivo (painel.ts, pedidos.ts...)
│  ├─ components/           ← pedaços reutilizáveis (kpiCard.ts...)
│  ├─ router.ts             ← rotas por hash: #/painel, #/pedidos
│  ├─ main.ts
│  └─ style.css
├─ .github/workflows/deploy.yml ← GitHub Actions → GitHub Pages
├─ index.html
├─ vite.config.ts           ← base: '/mvp-pedidos-compra/' + PWA
├─ package.json             ← scripts: dev, build, test
├─ requirements.txt         ← dependências Python
├─ .gitignore               ← node_modules, dist, .venv
└─ README.md

```

### O QUE NUNCA VAI PARA O GITHUB

`node_modules/` (centenas de MB, reinstalável com `npm ci`), `dist/` (gerado pelo build), `.venv/` (ambiente Python local) e qualquer arquivo com dado real. O `.gitignore` cuida disso: confirmam que ele existe **antes** do primeiro commit.

## PARTE 2

# Trabalhando com o agente

Como escrever um prompt que o agente acerta de primeira, o ciclo curto de cada tarefa e as regras de segurança do grupo.

1. Anatomia de um bom prompt de desenvolvimento
2. O ciclo de cada tarefa: planejar → gerar → testar → commitar
3. Regras de segurança ao trabalhar com o agente

§ 8

## Anatomia de um bom prompt de desenvolvimento

Na Aula 8 vocês viram os 4 princípios de prompt (papel, contexto, formato, iteração). Para programar, acrescentem mais dois: **restrições** e **critério de pronto**. Todo prompt deste caderno segue esta receita:

| Parte                  | O que escrever                            | Exemplo                                                      |
|------------------------|-------------------------------------------|--------------------------------------------------------------|
| 1. Contexto            | Citar os arquivos com @ para o agente ler | "Leia @docs/contexto.md e @docs/sprint.md"                   |
| 2. Tarefa              | UMA tarefa, com o código da sprint        | "Execute a tarefa S2.3 (cálculo do status do pedido)"        |
| 3. Onde                | Arquivos a criar ou alterar               | "Crie src/domain/regras.ts; não altere outros arquivos"      |
| 4. Restrições          | Stack, estilo, o que é proibido           | "TypeScript estrito, sem bibliotecas novas, sem dados reais" |
| 5. Pronto quando       | Como verificar que funcionou              | "npm test passa e a tela #/painel mostra 4 KPIs"             |
| 6. Formato da resposta | O que o agente deve devolver              | "Primeiro o plano; espere meu OK antes de codar"             |

### Modelo universal (copie e preencha) PAINEL AGENT DO ANTIGRAVITY

PROMPT

Contexto: leia @docs/contexto.md, @docs/sprint.md e @docs/REGRAS-DO-PROJETO.md.  
Tarefa: execute a tarefa [S?.?] do sprint.md: [DESCRIÇÃO CURTA].  
Onde: crie/altere somente [ARQUIVOS]. Não mexa em outros arquivos.  
Restrições: TypeScript estrito; sem bibliotecas novas sem me perguntar; somente dados sintéticos.  
Pronto quando: [CRITÉRIO VERIFICÁVEL, ex.: npm test passa e a tela X mostra Y].  
Resposta: primeiro me mostre o plano em tópicos e a lista de arquivos. Só escreva código depois do meu "OK".

### O @ É O SUPERPODER

Ao digitar @ na caixa do painel Agent, o Antigravity lista os arquivos do projeto. Mencionar o arquivo garante que o agente leia a versão atual dele, em vez de adivinhar. Sempre mencionem o contexto.md quando a tarefa envolver regra de negócio ou dado.

## O ciclo de cada tarefa: planejar → gerar → testar → commitar

O erro nº 1 com agentes de IA é pedir a sprint inteira de uma vez. O agente gera 30 arquivos, algo quebra, e ninguém sabe onde. Trabalhem em **ciclos curtos**: uma tarefa do sprint.md por ciclo, com um commit no final.



### 1 Pedir o plano

Use o modelo universal. Se o seu Antigravity mostrar os modos **Planning** e **Fast** no painel do agente, use **Planning** em tarefas novas: ele devolve um plano para você aprovar antes de mexer nos arquivos.

### 2 Aprovar ou corrigir

Leiam o plano em grupo. Ele cita arquivos que não deveria? Inventou uma regra que não está no contexto.md? Corrijam antes.

### 3 Gerar

O agente escreve o código. Quando ele pedir para rodar um comando no terminal, **leiam o comando antes de aprovar**.

### 4 Rodar e testar

`npm run dev` para ver a tela, `npm test` para os testes. Se der erro, copiem o erro inteiro para o agente (ver prompts de depuração).

### 5 Revisar o diff

No painel Source Control (Aula 6), cliquem em cada arquivo alterado: verde = linha nova, vermelho = linha removida. Algo estranho? Perguntem ao agente "por que você mudou isto?".

### 6 Commit + push

Uma mensagem por tarefa: `feat(pedidos): calcula status do pedido (S2.3)`. Depois, Sync/Push.

#### REGRA DOS 3 TESTES (AULA 7) ANTES DO COMMIT

Roda? (sem erro no terminal e no console do navegador) · Colega entende? (alguém do grupo explica o que o código faz) · Teste? (existe pelo menos um teste que prova que funciona). Faltou um: não commitem ainda.

## Regras de segurança ao trabalhar com o agente

| Situação                                                                                        | O que fazer                                                                                                                        |
|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| O agente quer rodar <code>rm, del, git reset --hard, --force</code> ou <code>--overwrite</code> | <b>Recusem</b> e perguntem por quê. Esses comandos apagam trabalho.                                                                |
| O agente alterou arquivos que não tinham nada a ver com a tarefa                                | No Source Control, descartem as mudanças desses arquivos ( <b>Discard Changes</b> ) e repitam o pedido dizendo "altere somente X". |
| O agente quer instalar uma biblioteca nova                                                      | Perguntem: "para que serve, qual a alternativa sem biblioteca, e ela funciona em site estático?"                                   |
| O agente sugere colocar uma chave de API, senha ou token no código                              | Recusem. No GitHub Pages tudo é público.                                                                                           |
| Vocês precisam de um exemplo com dados                                                          | Useem só dados sintéticos. Nunca coleem planilha real da SESP no chat.                                                             |
| A conversa ficou longa e o agente começou a "esquecer" coisas                                   | Abram uma conversa nova (+ no painel Agent) e comecem citando os arquivos com @.                                                   |
| O agente entrou em loop (erra, corrige, erra de novo)                                           | Parem. Voltem ao último commit bom (Discard Changes) e peçam uma abordagem diferente, mais simples.                                |

### TAREFAS GRANDES EM BRANCH SEPARADA

Antes de uma tarefa que mexe em muitos arquivos, criem uma branch (ex.: `sprint-3-telas`). Se der errado, a `main` e o site no ar continuam intactos. Detalhes na seção de versionamento.

## Sprint 0 — preparar o terreno

Instalar as ferramentas, criar o projeto sem perder o contexto.md e o sprint.md, publicar o repositório e fazer o agente provar que entendeu o projeto.

1. Passo 1 — Instalar Node.js e Python (e conferir o Git)
2. Passo 2 — Criar o projeto e trazer o contexto.md e o sprint.md
3. Passo 3 — Versionar e publicar o repositório no GitHub
4. Passo 4 — O primeiro prompt: o agente lê o contexto e o sprint.md
5. Passo 5 — O arquivo de regras do projeto (a "memória" do agente)
6. Passo 6 — Configurar o Vite para o GitHub Pages (o "base")

§ 11

### Passo 1 — Instalar Node.js e Python (e conferir o Git)

O Git vocês instalaram na Aula 6. Agora faltam duas ferramentas: o **Node.js**, que roda o TypeScript, o Vite e os testes, e o **Python**, que roda os scripts de dados.

#### 1 Node.js (versão LTS)

Baixe em [nodejs.org](https://nodejs.org) o instalador **LTS** para Windows e avance com as opções padrão. Ele instala junto o `npm`, o gerenciador de pacotes.

#### 2 Python 3.12 ou mais novo

Baixe em [python.org/downloads](https://python.org/downloads). Na primeira tela do instalador, **marque "Add python.exe to PATH"** antes de clicar em Install Now.

Esqueceu de marcar? Rode o instalador de novo, escolha Modify e marque a opção.

#### 3 Feche e reabra o Antigravity

O terminal só "enxerga" os programas novos depois de reaberto.

#### 4 Confira no terminal

Abra o terminal do Antigravity (**Terminal** → **New Terminal**) e rode os comandos abaixo. Cada um deve responder com um número de versão.

```
node -v          # ex.: v22.x ou v24.x
npm -v           # ex.: 10.x ou 11.x
python --version # ex.: Python 3.12.x
git --version    # ex.: git version 2.x
```

TERMINAL

#### ERRO: "NPM.PS1 NÃO PODE SER CARREGADO PORQUE A EXECUÇÃO DE SCRIPTS FOI DESABILITADA"

É uma proteção do PowerShell no Windows. Rode uma vez só o comando abaixo e responda S. Depois feche e reabra o terminal.

POWERSHELL

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

#### ERRO: "PYTHON" ABRE A MICROSOFT STORE

O Windows tem um atalho falso do Python. Vá em **Configurações** → **Aplicativos** → **Configurações avançadas de aplicativos** → **Aliases de execução de aplicativo** e desligue *python.exe* e *python3.exe*. Ou use `py` no lugar de `python`.

§ 12

## Passo 2 — Criar o projeto e trazer o contexto.md e o sprint.md

Vamos criar o esqueleto do aplicativo com o **Vite**, a ferramenta que roda o TypeScript no navegador durante o desenvolvimento e gera a versão final para publicar. O nome da pasta será o nome do repositório e o final do endereço do site, então escolham bem: **minúsculas, sem espaço, sem acento, palavras separadas por hífen**.

### 1 Abra o terminal na pasta onde ficam seus projetos

Ex.: `D:\Dev`. No terminal: `cd D:\Dev`

### 2 Crie o projeto

Rode o comando abaixo, trocando pelo nome do projeto de vocês. `vanilla-ts` = TypeScript puro, sem framework.

Se o `sprint.md` de vocês definiu React, troquem por `react-ts`. Todo o resto do caderno funciona igual.

### 3 Instale as dependências

Entre na pasta e rode `npm install`. Demora 1 a 2 minutos na primeira vez.

### 4 Traga os dois arquivos do NotebookLM

Crie a pasta `docs` dentro do projeto e mova para ela o `contexto.md` e o `sprint.md`.

### 5 Abra a pasta no Antigravity

**File** → **Open Folder** e escolham a pasta do projeto. Na barra lateral devem aparecer `docs/`, `src/`, `index.html`, `package.json`.

### 6 Teste

Rode `npm run dev` e abram o endereço que aparecer (ex.: `http://localhost:5173/`). Apareceu a página de boas-vindas do Vite? O terreno está pronto. `Ctrl+C` no terminal para parar.

```
cd D:\Dev
npm create vite@latest mvp-pedidos-compra -- --template vanilla-ts
cd mvp-pedidos-compra
npm install
mkdir docs
# mova contexto.md e sprint.md para a pasta docs/ (no Explorer do Antigravity: arrastar e soltar)
npm run dev
```

### NÃO CRIEM O PROJETO VITE DENTRO DA PASTA ONDE JÁ ESTÃO O CONTEXTO.MD E O SPRINT.MD

Numa pasta que já tem arquivos, o criador do Vite cancela. Se o agente sugerir a opção `--overwrite`, ela **apaga os arquivos que já estão lá**, incluindo os dois `.md`. Por isso o caminho seguro é: criar o projeto numa pasta nova e só depois mover os arquivos para dentro dela.

### 💡 Prefere que o agente conduza?

#### PAINEL AGENT DO ANTIGRAVITY

Estou no Windows, com o terminal na pasta D:\Dev. Quero criar um projeto Vite com TypeScript chamado mvp-pedidos-compra.

Me diga os comandos, um por vez, e explique em 1 linha o que cada um faz. Não use `--overwrite` e não apague nenhum arquivo.

§ 13

## Passo 3 — Versionar e publicar o repositório no GitHub

Agora que a pasta existe, ela vira um repositório Git publicado no GitHub, com o mesmo fluxo do Laboratório 1 da Aula 6.

- 1 **Confira o `.gitignore`**  
O Vite já cria um `.gitignore` com `node_modules` e `dist`. Acrescentem no final as linhas do bloco abaixo, para o Python.
- 2 **Inicialize e publique**  
No painel **Source Control**, clique em **Publish to GitHub** e escolha **Public repository**. O GitHub Pages gratuito exige repositório público.  
Se aparecer "Initialize Repository" primeiro, clique nele, faça o primeiro commit e depois publique.
- 3 **Primeiro commit**  
Mensagem: `chore: estrutura inicial do projeto (Vite + TS) e documentos do NotebookLM`.
- 4 **Confira no GitHub**  
Abra `https://github.com/SEU-USUARIO/mvp-pedidos-compra`: devem aparecer `docs/`, `src/` e os arquivos do Vite, mas **não** a pasta `node_modules`.

```
# Python
.venv/
__pycache__/
*.pyc
.pytest_cache/
```

### UM REPOSITÓRIO POR GRUPO

Um integrante cria e publica. Os demais entram como colaboradores: **Settings** → **Collaborators** → **Add people** no GitHub. Depois cada um clona (Aula 6, Cenário A) e roda `npm install` na própria máquina.

§ 14

## Passo 4 — O primeiro prompt: o agente lê o contexto e o sprint.md

Antes de escrever qualquer linha de código, façam o agente **provar que entendeu** o projeto. Esse prompt não gera código: gera um diagnóstico para vocês conferirem.

### Prompt 0.1 — Diagnóstico do projeto PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia com atenção @docs/contexto.md e @docs/sprint.md. NÃO escreva código ainda.  
Me devolva, em Markdown:

1. O problema e a solução em 3 linhas, com as suas palavras.
2. A lista de telas do MVP e qual papel (Admin, Editor, Leitor, Aprovador) usa cada uma.
3. As entidades de dados com seus campos e tipos, em formato de tabela.
4. As regras de negócio que você identificou, numeradas (RN01, RN02...).
5. Os campos sensíveis (LGPD) e como devem ser mascarados.
6. Uma tabela "item do sprint.md | como fica num site estático no GitHub Pages", apontando o que precisa ser adaptado.
7. Contradições ou lacunas entre os dois arquivos e as perguntas que você faria ao grupo antes de começar.

Leiam a resposta em grupo. Respondam às perguntas do item 7 e, se o item 6 apontou adaptações, peçam a versão ajustada do plano:

### Prompt 0.2 — Adaptar o sprint.md ao GitHub Pages PAINEL AGENT DO ANTIGRAVITY

PROMPT

Com as respostas abaixo, reescreva o @docs/sprint.md adaptado para o GitHub Pages (site estático):

- Backend/API vira a camada src/services em TypeScript lendo public/data/\*.json.
- Banco de dados vira JSON gerado por scripts/gerar\_dados.py (Python) + IndexedDB/localStorage para cadastros.
- Login vira seletor de papel (RBAC simulado).

Mantenha a numeração das tarefas (S1.1, S1.2...) e acrescente em cada tarefa: arquivos, linguagem e critério de aceite.

Salve em docs/sprint.md e me mostre um resumo do que mudou.

Respostas do grupo: **[COLE AQUI AS RESPOSTAS ÀS PERGUNTAS DO ITEM 7]**

### COMMIT DESTA ETAPA

docs: diagnóstico do agente e sprint.md adaptado ao GitHub Pages

## Passo 5 — O arquivo de regras do projeto (a "memória" do agente)

O agente não lembra das conversas anteriores quando vocês abrem uma nova. A solução é um arquivo com as regras permanentes do projeto, mencionado com @ nos prompts. Peçam para o agente criá-lo:

### Prompt 0.3 — Criar as regras do projeto PAINEL AGENT DO ANTIGRAVITY

PROMPT

```
Crie o arquivo docs/REGRAS-DO-PROJETO.md com as regras permanentes deste projeto, baseadas em
@docs/contexto.md e @docs/sprint.md.
Inclua, em tópicos curtos:
- Stack: Vite + TypeScript estrito, PWA com vite-plugin-pwa, Python 3.12 só em scripts/, testes com Vitest e
  pytest.
- Hospedagem: GitHub Pages, site estático. Base do Vite: '/' [NOME-DO-REPO] '/'. Rotas por hash (#/tela).
- Caminhos de dados sempre com import.meta.env.BASE_URL, nunca "/data/..." fixo.
- Estrutura de pastas: src/domain, src/auth, src/services, src/pages, src/components, scripts, public/data.
- Dados: somente sintéticos; CPF e nomes sempre mascarados no Python antes de gerar o JSON.
- Proibido: dados reais, senhas, chaves de API, bibliotecas novas sem aprovação, apagar arquivos sem pedir.
- Código e comentários em português; nomes de variáveis em português sem acento (ex.: valorEstimado).
- Cada tarefa termina com testes passando e uma sugestão de mensagem de commit (Conventional Commits).
- Glossário do domínio: [TERMOS DO PROJETO, ex.: pedido, cotação, auditoria].
Máximo de 1 página.
```

#### COMO USAR DAQUI PARA FRENTE

Comecem toda conversa nova com: *"Leia @docs/REGRAS-DO-PROJETO.md e @docs/contexto.md antes de qualquer coisa."* Se o agente desobedecer uma regra, respondam citando a regra. Se o Antigravity de vocês tiver a opção de **regras do workspace** nas configurações do agente, podem colar o conteúdo lá também.

## Passo 6 — Configurar o Vite para o GitHub Pages (o "base")

O site de vocês não vai ficar na raiz do domínio, mas numa subpasta: `https://usuario.github.io/mvp-pedidos-compra/`. Se o Vite não souber disso, o site publicado abre **em branco**, porque procura os arquivos no lugar errado. É o erro mais comum de todos.

VITE.CONFIG.TS

```
import { defineConfig } from 'vite'

// Troque pelo nome EXATO do seu repositório no GitHub (maiúsculas e minúsculas importam)
const REPO = 'mvp-pedidos-compra'

export default defineConfig({
  base: `/${REPO}/`,
})
```

E, em todo lugar que o código buscar um arquivo de dados, usem o `BASE_URL` que o Vite preenche sozinho:

```
// X ERRADO: funciona no computador, quebra no GitHub Pages
const resp = await fetch('/data/pedidos.json')

// ✓ CERTO: vira /mvp-pedidos-compra/data/pedidos.json no site publicado
const resp = await fetch(`${import.meta.env.BASE_URL}data/pedidos.json`)
```

#### Prompt 0.4 — Configurar o base

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Crie o arquivo vite.config.ts com base: `'/[NOME-DO-REPO]'` para publicar no GitHub Pages. Depois procure em todo o `src/` qualquer `fetch` ou caminho que comece com `"/` e troque para usar `import.meta.env.BASE_URL`. Me mostre a lista do que alterou.

#### COMMIT

```
chore: configura base do Vite para o GitHub Pages
```

## Construindo sprint a sprint

Dados sintéticos em Python, regras de negócio em TypeScript, telas, PWA, homologação e deploy automático. Cada sprint tem os prompts prontos, o gabarito do que conferir e as mensagens de commit.

1. O mapa das 5 sprints do MVP
2. Sprint 1 — Modelagem e dados sintéticos com Python
3. Sprint 2 — Regras de negócio, papéis e auditoria em TypeScript
4. Sprint 3 — Telas, painel e navegação
5. Sprint 4a — Transformar em PWA (instalável e offline)
6. Sprint 4b — Homologação: testes, LGPD e revisão
7. Sprint 5 — Deploy automático no GitHub Pages com GitHub Actions
8. Versionamento profissional: branches, commits, PRs e tags
9. README e transparência: documentando o que a IA fez

§ 17

### O mapa das 5 sprints do MVP

As sprints abaixo seguem a sugestão da Aula 7, adaptadas para TS + Python + PWA no GitHub Pages. Se o sprint.md de vocês tem outra divisão, vale o de vocês: usem os prompts das próximas seções como modelo, trocando os códigos das tarefas.

**1**

#### Sprint 1 · Fundação e dados

Python + TS · entregável: JSON sintéticos gerados e tipos definidos

- Tipos das entidades em `src/domain/tipos.ts` ;
- `scripts/gerar_dados.py` com dados sintéticos e máscara LGPD;
- testes pytest garantindo que nenhum dado sensível sai sem máscara.

**2**

#### Sprint 2 · Regras de negócio, papéis e auditoria

TS · entregável: regras testadas com Vitest

- Camada `src/services/api.ts` lendo os JSON;
- regras RN01, RN02... em `src/domain/regras.ts` ;
- RBAC simulado ( `src/auth/papeis.ts` ) e log de auditoria local.

3

### Sprint 3 · Telas e navegação

TS + CSS · entregável: telas navegáveis no npm run dev

- Rotas por hash, layout responsivo;
- painel com KPIs e gráfico, lista com filtros, formulário com validação;
- teste dos 5 segundos (o KPI principal salta aos olhos).

4

### Sprint 4 · PWA e homologação

TS + JS · entregável: app instalável, offline e revisado

- vite-plugin-pwa, manifesto e ícones;
- testes por papel, varredura LGPD, Lighthouse;
- revisão de código pelo agente e pelo grupo.

5

### Sprint 5 · Deploy e documentação

YAML + Markdown · entregável: link público funcionando

- GitHub Actions publicando no GitHub Pages a cada push;
- README com roadmap, transparência de IA e LGPD;
- tag v1.0.0 e o pitch de 3 minutos.

§ 18

## Sprint 1 — Modelagem e dados sintéticos com Python

#### QUANDO

Primeira sprint: antes de qualquer tela.

#### POR QUE

Tela sem dado não se testa. Com dados sintéticos realistas, vocês veem o painel "vivo" desde o início.

#### EXEMPLO NO MVP

120 pedidos de compra com protocolo, unidade, valor, status e CPF do solicitante mascarado.

### 1.1 · Tipos das entidades (TypeScript)



#### Prompt S1.1 — Tipos

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/REGRAS-DO-PROJETO.md e a seção de entidades de @docs/contexto.md.

Tarefa S1.1: crie src/domain/tipos.ts com uma interface TypeScript para cada entidade (ex.: Pedido, Unidade, Cotacao).

- Use tipos union para campos de lista fechada (ex.: status: 'rascunho' | 'enviado' | 'aprovado').
- Datas como string ISO (AAAA-MM-DD).
- Comente cada campo em 1 linha, em português.

Pronto quando: npx tsc --noEmit roda sem erros.

## 1.2 · Ambiente Python (uma vez por computador)

TERMINAL

```
python -m venv .venv
.venv\Scripts\Activate.ps1    # o terminal passa a mostrar (.venv) no começo da linha
pip install pytest
pip freeze > requirements.txt
```

## 1.3 · Script gerador de dados sintéticos

### Prompt S1.2 — Gerador de dados

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/REGRAS-D0-PROJETO.md, @docs/contexto.md e @src/domain/tipos.ts.

Tarefa S1.2: crie scripts/gerar\_dados.py (Python 3.12, somente biblioteca padrão) que gera dados SINTÉTICOS e grava um JSON por entidade em public/data/ (ex.: public/data/pedidos.json).

Requisitos:

- random.seed(42) para os dados serem sempre os mesmos (reprodutível).
- Os campos do JSON devem ter EXATAMENTE os mesmos nomes das interfaces de tipos.ts.
- Volume: [QUANTIDADE, ex.: 120 pedidos] distribuídos ao longo de 2026.
- Realismo de Mato Grosso: unidades e municípios reais (Cuiabá, Várzea Grande, Rondonópolis...), valores plausíveis.
- Inclua casos de borda: [EX.: 5 pedidos acima de R\$ 50 mil, 3 devolvidos, 2 com campo opcional vazio].
- CPF: gere CPF matematicamente válido e grave SOMENTE mascarado no formato \*\*\*.123.\*\*\*-\*\* (LGPD).
- Nomes de pessoas: use nomes fictícios ou só a função (ex.: "Solicitante 014").
- json.dumps com ensure\_ascii=False e encoding utf-8.
- No final, imprima quantos registros gerou por arquivo.

Pronto quando: python scripts/gerar\_dados.py roda e os arquivos aparecem em public/data/.

Um trecho do que o script deve ter, para vocês conferirem na revisão:

PYTHON

```
random.seed(42) # mesma semente = mesmos dados sempre

def mascarar_cpf(cpf: str) → str:
    return f"***.{cpf[3:6]}.***-**"

pedidos.append({
    "protocolo": f"PC-2026-{i:04d}",
    "unidade": random.choice(UNIDADES),
    "solicitante_cpf": mascarar_cpf(cpf_valido()), # nunca o CPF completo
    "valor_estimado": round(random.uniform(800, 95000), 2),
    "status": random.choice(STATUS),
})
```

## 1.4 · Teste de LGPD com pytest

### Prompt S1.3 — Teste LGPD PAINEL AGENT DO ANTIGRAVITY

PROMPT

Tarefa S1.3: crie `scripts/test gerar_dados.py` com `pytest` que:

1. executa `scripts/gerar_dados.py`;
2. falha se encontrar em qualquer arquivo de `public/data/` um CPF completo (regex de 11 dígitos, com ou sem pontuação);
3. confere a quantidade de registros e que todo registro tem os campos obrigatórios de `@src/domain/tipos.ts`;
4. confere as regras de borda que pedi (ex.: existem pedidos acima de R\$ 50 mil).

Pronto quando: `python -m pytest -q scripts` mostra todos os testes passando.

TERMINAL

```
python scripts/gerar_dados.py
python -m pytest -q scripts
```

#### COMMITTS DA SPRINT 1

`feat(dominio): tipos das entidades (S1.1)` · `feat(dados): gerador de dados sintéticos (S1.2)` · `test(lgpd): garante CPF mascarado nos JSON (S1.3)`

§ 19

## Sprint 2 — Regras de negócio, papéis e auditoria em TypeScript

Esta sprint é o "backend" do MVP, rodando no navegador. A regra de ouro: regra de negócio não fica dentro de tela. Ela fica em funções puras em `src/domain/`, fáceis de testar.

### 2.1 · A camada de serviços (a "API" do MVP)

#### Prompt S2.1 — Serviços PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia `@docs/REGRAS-DO-PROJETO.md` e `@src/domain/tipos.ts`.

Tarefa S2.1: crie `src/services/api.ts` com funções assíncronas que imitam uma API REST: `listarPedidos()`, `obterPedido(id)`, `listarUnidades()` **[AJUSTE PARA AS ENTIDADES DO PROJETO]**.

- Leia os JSON com `fetch(`${import.meta.env.BASE_URL}data/<arquivo>.json`)`.
- Trate erro de rede com mensagem clara em português.
- Guarde em memória o que já foi carregado (não buscar o mesmo JSON duas vezes).
- Escreva as funções contra uma interface `FonteDeDados`, para no futuro trocar JSON por uma API real sem mudar as telas.

## 2.2 · Regras de negócio com teste

### Prompt S2.2 — Regras + Vitest

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia as regras de negócio (RN01, RN02...) em @docs/contexto.md.  
Tarefa S2.2: implemente cada regra como função pura em src/domain/regras.ts (sem acessar tela nem fetch).  
Crie src/domain/regras.test.ts com Vitest: pelo menos 2 testes por regra, incluindo o caso de limite (ex.: exatamente R\$ 50.000,00 NÃO exige auditoria; R\$ 50.000,01 exige).  
Adicione ao package.json o script "test": "vitest run" e instale o vitest como devDependency.  
Pronto quando: npm test mostra todos os testes passando. Me mostre a saída.

TS

```
export const LIMITE_AUDITORIA = 50000

/** RN03 - pedido acima de R$ 50 mil exige auditoria da Controladoria */
export function exigeAuditoria(p: Pedido): boolean {
  return p.valor_estimado > LIMITE_AUDITORIA
}
```

## 2.3 · Papéis e permissões (RBAC simulado)

### Prompt S2.3 — Papéis

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia a tabela de papéis em @docs/contexto.md.  
Tarefa S2.3: crie src/auth/papeis.ts com:  
- type Papel = 'ADMIN' | 'EDITOR' | 'LEITOR' | 'APROVADOR' e type Acao = 'ver' | 'criar' | 'editar' | 'excluir' | 'aprovar' | 'ver\_log';  
- uma tabela de permissões por papel, igual à do contexto.md;  
- função pode(papel, acao): boolean;  
- papel atual guardado no localStorage, com getPapelAtual() e setPapelAtual().  
Crie testes Vitest que conferem cada linha da tabela de permissões.  
Deixe claro em comentário que isto é uma SIMULAÇÃO para demonstração, não segurança real.

## 2.4 · Log de auditoria e armazenamento local

### Prompt S2.4 — Auditoria

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Tarefa S2.4: crie src/services/auditoria.ts que registra cada ação relevante (criar, editar, excluir, aprovar) com: dataHora ISO, papel, acao, entidade, id, valorAntes, valorDepois.  
- Guarde no IndexedDB (sem biblioteca externa) ou, se for mais simples, no localStorage. Explique a escolha.  
- Função exportarCsv() que baixa o log como arquivo .csv (separador ;, UTF-8 com BOM para abrir certo no Excel).  
- O log não pode ser apagado pelo papel que gerou a ação (regra da Aula 7).  
Crie também src/services/armazenamento.ts para salvar os pedidos criados/editados pelo usuário, mesclando com os dados do JSON na hora de listar.

### COMMITTS DA SPRINT 2

feat(api): camada de serviços sobre JSON (S2.1) · feat(regras): RN01-RN0X com testes (S2.2) ·  
feat(auth): RBAC simulado (S2.3) · feat(auditoria): log local com exportação CSV (S2.4)

## Sprint 3 — Telas, painel e navegação

Agora o MVP ganha cara. Lembrem das aulas de visualização: **título que diz a conclusão**, eixo começando no zero, cor com significado (vermelho = crítico, verde = meta), nada de pizza 3D.

### 3.1 · Rotas e layout

#### Prompt S3.1 — Rotas e layout

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/REGRAS-DO-PROJETO.md e a lista de telas em @docs/contexto.md.  
 Tarefa S3.1: crie src/router.ts com rotas por hash (#/painel, #/pedidos, #/pedidos/novo, #/auditoria) **[AJUSTE ÀS TELAS]**.  
 - Rota inicial: #/painel. Rota desconhecida: tela "Página não encontrada" com link para o painel.  
 - Layout com cabeçalho (nome do sistema + seletor "Ver como: [papel]"), menu e área de conteúdo.  
 - Cada tela em um arquivo de src/pages/ exportando uma função render(container).  
 - Esconder do menu as telas que o papel atual não pode ver (use pode() de src/auth/papeis.ts).  
 - CSS em src/style.css, mobile first, cores com variáveis CSS, contraste AA.  
 Por que hash: o GitHub Pages não conhece rotas como /pedidos e devolveria erro 404 ao recarregar a página.

### 3.2 · O painel do gestor

#### Prompt S3.2 — Painel

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Tarefa S3.2: crie src/pages/painel.ts, a tela principal para o **[PAPEL, ex.: Secretário]**.  
 - 4 cartões de KPI no topo, com os KPIs de @docs/contexto.md (valor grande + rótulo + comparação com a meta).  
 - 1 gráfico de barras com Chart.js (instale chart.js) mostrando **[MÉTRICA POR CATEGORIA]**, eixo Y começando em zero, título que afirma a conclusão (ex.: "Compras concentra 40% dos pedidos em atraso").  
 - 1 tabela com os 10 itens que exigem atenção (ex.: aguardando auditoria há mais de 5 dias).  
 - Tudo calculado com as funções de src/domain/regras.ts (nada de cálculo solto na tela).  
 Pronto quando: em até 5 segundos alguém de fora do grupo diz qual é o indicador principal.

### 3.3 · Lista com filtros e formulário

#### Prompt S3.3 — Lista

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Tarefa S3.3: crie src/pages/pedidos.ts com a lista de **[ENTIDADE]**:  
 busca por texto, filtros por **[CAMPOS, ex.: unidade e status]**, ordenação por coluna e paginação de 20 em 20. Os filtros ficam na URL (ex.: #/pedidos?status=aprovado) para o link poder ser compartilhado.  
 Botões Editar/Excluir/Aprovar aparecem somente se pode(papelAtual, acao) for verdadeiro.

Tarefa S3.4: crie `src/pages/formulario.ts` para criar e editar `[ENTIDADE]`.

- Validação em português ao sair do campo e ao salvar (obrigatórios, valores mínimos, datas coerentes).
- Campos sensíveis com máscara visual (ex.: CPF exibido como `***.123.**-*`).
- Ao salvar: grava via `src/services/armazenamento.ts` e registra no log de auditoria.
- Acessibilidade: label em todo campo, navegação completa por teclado, mensagem de erro ligada ao campo (`aria-describedby`).

#### VIU ALGO FEIO? MOSTRE PARA O AGENTE

Tirem um print da tela (`Win+Shift+S`) e cole no painel Agent com o pedido: *"Nesta tela, o cartão de KPI está cortado no celular. Corrija somente o CSS em `src/style.css`."* Imagem + pedido específico funcionam muito melhor que descrever.

#### COMMITTS DA SPRINT 3

`feat(ui): rotas por hash e layout (S3.1)` · `feat(painel): KPIs e gráfico (S3.2)` · `feat(pedidos): lista com filtros (S3.3)` · `feat(pedidos): formulário com validação (S3.4)`

§ 21

## Sprint 4a — Transformar em PWA (instalável e offline)

PWA (*Progressive Web App*) é um site que se comporta como aplicativo: tem ícone na tela inicial do celular, abre em tela cheia e funciona sem internet. Duas peças fazem isso: o **manifesto** (nome, ícones, cores) e o **service worker** (um JavaScript que guarda os arquivos no aparelho). O plugin `vite-plugin-pwa` gera os dois.

### 1 Instale o plugin e o gerador de ícones

Comandos no bloco logo abaixo.

### 2 Crie o ícone-fonte

Um arquivo `public/icon.svg` quadrado (peçam ao agente um SVG simples com as iniciais do sistema).

### 3 Gere os ícones PNG

O gerador cria os tamanhos 64, 192, 512, o ícone "maskable" (Android), o do iPhone e o favicon.

### 4 Configure o `vite.config.ts`

Com o bloco completo abaixo.

### 5 Teste

`npm run build` e depois `npm run preview`. Abram o endereço mostrado, depois **F12** → **Application** → **Manifest** (ícones sem erro) e **Service Workers** (status *activated*). Na barra de endereço deve aparecer o botão de instalar.

```
npm install -D vite-plugin-pwa @vite-pwa/assets-generator
npx pwa-assets-generator --preset minimal-2023 public/icon.svg
```

```
import { defineConfig } from 'vite'
import { VitePWA } from 'vite-plugin-pwa'

const REPO = 'mvp-pedidos-compra' // nome EXATO do repositório

export default defineConfig({
  base: `/${REPO}/`,
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon-180x180.png'],
      manifest: {
        name: 'Portal de Pedidos de Compra – SESP/MT (MVP)',
        short_name: 'Pedidos SESP',
        description: 'MVP acadêmico com dados sintéticos – Curso SESP/MT',
        lang: 'pt-BR',
        theme_color: '#1f4b8f',
        background_color: '#ffffff',
        display: 'standalone',
        start_url: '.',
        scope: '.',
        icons: [
          { src: 'pwa-64x64.png', sizes: '64x64', type: 'image/png' },
          { src: 'pwa-192x192.png', sizes: '192x192', type: 'image/png' },
          { src: 'pwa-512x512.png', sizes: '512x512', type: 'image/png' },
          { src: 'maskable-icon-512x512.png', sizes: '512x512',
            type: 'image/png', purpose: 'maskable' },
        ],
      },
    },
  ],
  workbox: {
    // incluí os JSON de public/data: o app abre offline com os dados
    globPatterns: ['**/*.{js,css,html,svg,png,ico,json}'],
  },
})
```

#### POR QUE START\_URL E SCOPE SÃO "."

O ponto significa "a pasta onde está o manifesto", ou seja, `/mvp-pedidos-compra/`. Se colocarem `"/"`, o app instalado abre a raiz de `github.io` e dá erro 404.

Leia @vite.config.ts e @docs/REGRAS-DO-PROJETO.md.

Tarefa S4.1: transforme o projeto em PWA com vite-plugin-pwa:

- manifest com name "[NOME COMPLETO]", short\_name "[NOME CURTO até 12 letras]", lang pt-BR, theme\_color [COR], display standalone, start\_url "." e scope "." (o site fica em subpasta do GitHub Pages);
- crie public/icon.svg (quadrado 512x512, fundo [COR], iniciais "[SIGLA]" em branco) e gere os PNG com npx pwa-assets-generator --preset minimal-2023 public/icon.svg;
- service worker com registerType autoUpdate, cacheando também os JSON de public/data para funcionar offline;
- quando houver versão nova, mostrar um aviso discreto "Nova versão disponível – recarregar".

Pronto quando: npm run build && npm run preview abre o site, o DevTools mostra o manifesto sem erros e, com a rede desligada (DevTools → Network → Offline), a página recarrega com os dados.

#### DURANTE O DESENVOLVIMENTO O SERVICE WORKER PODE "PRENDER" VERSÕES ANTIGAS

Se a tela não mudar depois de um build novo: **F12 → Application → Service Workers → Unregister e Ctrl+Shift+R**. No celular com o app instalado: fechem e abram o app duas vezes (o *autoUpdate* troca a versão na segunda abertura).

§ 22

## Sprint 4b — Homologação: testes, LGPD e revisão

Homologar é provar, antes do público ver, que o sistema faz o que o contexto.md promete. Quatro frentes:

| Frente              | Como verificar                                                  | Ferramenta                                  |
|---------------------|-----------------------------------------------------------------|---------------------------------------------|
| Regras de negócio   | Todos os testes passando                                        | npm test (Vitest)                           |
| Dados e LGPD        | Nenhum dado sensível sem máscara nos JSON nem no site compilado | pytest + prompt de varredura                |
| Papéis              | Cada papel vê e faz só o que pode                               | Roteiro manual (prompt abaixo)              |
| Qualidade da página | Desempenho, acessibilidade, boas práticas                       | <b>F12 → Lighthouse</b> (meta: acima de 90) |

#### Prompt S4.2 — Roteiro de testes por papel

Leia @docs/contexto.md e @src/auth/papeis.ts.

Gere um roteiro de teste manual em docs/roteiro-homologacao.md: para cada papel (Admin, Editor, Leitor, Aprovador), uma tabela Passo | Ação | Resultado esperado, cobrindo: telas visíveis, botões visíveis, criar, editar, aprovar, excluir e ver o log. Inclua 3 casos de erro (ex.: Leitor tentando acessar #/pedidos/novo pela URL).

Faça uma varredura de LGPD no projeto:

1. Rode `npm run build` e procure em `dist/` e em `public/data/` qualquer CPF completo, e-mail, telefone ou nome que pareça real.
2. Procure no código chaves de API, tokens, senhas ou URLs internas da SESP.
3. Confira se o README avisa que os dados são sintéticos.

Me devolva uma tabela Achado | Arquivo | Linha | Gravidade | Correção sugerida. Não corrija nada ainda.

Atue como revisor de código sênior. Revise os arquivos alterados nesta sprint (`@src/domain`, `@src/pages`, `@src/services`).

Procure: bugs, regra de negócio implementada diferente do `@docs/contexto.md`, cálculo feito dentro de tela, texto sem acento ou em inglês para o usuário, acessibilidade (labels, contraste, teclado) e código duplicado.

Responda em tabela Problema | Arquivo:linha | Por que importa | Correção. Ordene do mais grave ao menos grave.

Não altere nada: eu escolho o que corrigir.

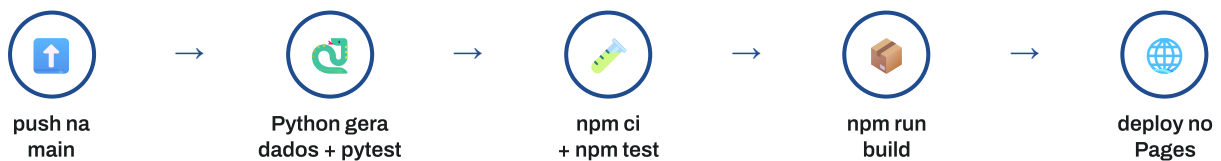
#### COMMITTS DA SPRINT 4

feat(pwa): manifesto, ícones e offline (S4.1) · docs: roteiro de homologação (S4.2) · fix: correções da revisão e da varredura LGPD (S4.3/S4.4)

## § 23

### Sprint 5 — Deploy automático no GitHub Pages com GitHub Actions

O GitHub Actions é um computador do GitHub que, a cada `push`, baixa o código, gera os dados com Python, roda os testes, faz o build e publica no Pages. Se um teste falhar, **nada é publicado**, e o site no ar continua na última versão boa.



#### 1 Ative o Pages com Actions (uma vez)

No GitHub: repositório → Settings → Pages → Build and deployment → Source: GitHub Actions.

#### 2 Crie o arquivo do workflow

Caminho exato: `.github/workflows/deploy.yml` (com o ponto no começo de `.github`).

#### 3 Confira `requirements.txt` e `package-lock.json`

Os dois precisam estar commitados: o Actions instala exatamente o que está neles.

4

## Commit e push na main

ci: deploy automático no GitHub Pages

5

## Acompanhe

Aba **Actions** do repositório: bolinha amarela = rodando, verde = publicado, vermelha = falhou (cliquem para ver o passo que quebrou).

6

## Abra o site

<https://SEU-USUARIO.github.io/mvp-pedidos-compra/>. O endereço também aparece em Settings → Pages e no resumo do workflow.

.GITHUB/WORKFLOWS/DEPLOY.YML

```
# Publica o MVP no GitHub Pages a cada push na branch main.
# Pré-requisito (uma vez só): Settings → Pages → Source: "GitHub Actions".
name: Deploy no GitHub Pages

on:
  push:
    branches: [main]
    workflow_dispatch:      # botão "Run workflow" na aba Actions

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Python – gerar dados sintéticos e testar LGPD
        uses: actions/setup-python@v7
        with:
          python-version: '3.12'
        - run: pip install -r requirements.txt
        - run: python scripts/gerar_dados.py
        - run: python -m pytest -q scripts

      - name: Node – instalar, testar e compilar o front-end
        uses: actions/setup-node@v7
        with:
          node-version: '22'
        - run: npm ci
        - run: npm test
        - run: npm run build

      - uses: actions/configure-pages@v6
      - uses: actions/upload-pages-artifact@v5
        with:
```

```

    path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - id: deployment
        uses: actions/deploy-pages@v5

```

#### VALIDADO

Este workflow e o `vite.config.ts` da Sprint 4a foram testados num projeto de referência: build, testes Python e Vitest, site servido na subpasta do repositório, navegação por hash e funcionamento offline. As versões das actions ( `checkout@v7` , `deploy-pages@v5` ...) eram as mais recentes em setembro de 2026. Se o agente sugerir uma versão mais nova, podem aceitar.

#### Prompt S5.1 — Workflow de deploy

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/REGRAS-D0-PROJETO.md, @package.json e @vite.config.ts.  
 Tarefa S5.1: crie .github/workflows/deploy.yml para publicar no GitHub Pages a cada push na main:  
 job build (checkout; Python 3.12 com pip install -r requirements.txt, python scripts/gerar\_dados.py e pytest;  
 Node 22 com npm ci, npm test e npm run build; upload-pages-artifact com path dist) e job deploy com deploy-pages.  
 Permissões: contents read, pages write, id-token write. Use as versões mais recentes das actions oficiais.  
 Depois me diga, passo a passo, o que devo configurar em Settings → Pages no GitHub.

#### Prompt S5.2 — Quando o deploy falhar

PAINEL AGENT DO ANTIGRAVITY

PROMPT

O workflow do GitHub Actions falhou. Abaixo está o log do passo que ficou vermelho.  
 Explique a causa em linguagem simples, diga em qual arquivo está o problema e proponha a correção mínima.  
 Log: [COLE AQUI O TRECHO DO LOG, a partir da primeira linha com "Error"]

#### O SITE ABRIU EM BRANCO?

Em 9 de cada 10 casos é o `base` do `vite.config.ts` diferente do nome do repositório (atenção a maiúsculas). Abram F12 → Console: erros 404 em arquivos `/assets/...` confirmam o diagnóstico.

§ 24

## Versionamento profissional: branches, commits, PRs e tags

Com o deploy automático, a `main` é o que está no ar. Por isso ela só recebe código testado. O trabalho do dia a dia acontece em **branches**.



| Prática                            | Como fazer no Antigravity/GitHub                                                              | Exemplo                                                          |
|------------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Branch por sprint ou tarefa grande | Barra inferior (nome da branch) → <b>Create new branch</b>                                    | sprint-2-regras, fix-grafico-mobile                              |
| Conventional Commits               | Prefixo que diz o tipo da mudança                                                             | feat: nova função · fix: correção · test: · docs: · ci: · chore: |
| Pull Request (PR)                  | GitHub → aba <b>Pull requests</b> → <b>New</b> (ou o botão que aparece após o push da branch) | "Sprint 2: regras de negócio e papéis"                           |
| Revisão em dupla                   | Outro integrante lê o PR, testa localmente e aprova                                           | Comentário: "testei como Leitor, OK"                             |
| Tag de versão                      | Ao fim de cada sprint: GitHub → <b>Releases</b> → <b>Draft a new release</b>                  | v0.1.0 (Sprint 1) ... v1.0.0 (MVP)                               |
| Issues                             | Uma issue por tarefa do sprint.md; o commit cita o número                                     | feat(painel): KPIs (S3.2) #12                                    |

### Prompt V1 — Mensagens de commit

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Analise as mudanças ainda não commitadas (git diff) e sugira:

1. se devem virar 1 commit ou vários (agrupe por assunto);
2. a mensagem de cada commit no padrão Conventional Commits, em português, citando o código da tarefa do sprint.md.

Não faça o commit: só me mostre as sugestões.

### Prompt V2 — Descrição de Pull Request

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/sprint.md e gere a descrição de um Pull Request da branch atual para a main, com:

Resumo (3 linhas), Tarefas concluídas (com códigos S?.?), Como testar (passos para o revisor), Prints sugeridos, Checklist (testes passando, LGPD verificada, sem dados reais).

### Prompt V3 — Issues a partir do sprint.md

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/sprint.md e transforme cada tarefa em uma issue do GitHub:

título "[S?.?]" descrição curta", corpo com critério de aceite em checklist e rótulo (sprint-1, sprint-2...).

Entregue em Markdown para eu colar uma a uma no GitHub.

## README e transparência: documentando o que a IA fez

Na Aula 7 ficou combinado: o README lista o que a IA gerou, quais prompts foram usados e o que foi revisado por pessoas. Isso é **maturidade técnica**, não confissão. Numa auditoria, é o que prova que houve controle humano.

### Prompt R1 — README completo

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/contexto.md, @docs/sprint.md e a estrutura do projeto.  
Escreva o README.md do repositório, em português, com as seções:

1. Nome do sistema + link do site no GitHub Pages + print da tela principal (deixe o marcador da imagem).
2. O problema e a solução (3 linhas) e os KPIs.
3. Como usar: papéis disponíveis no seletor "Ver como" e o que cada um faz.
4. Tecnologias: TypeScript + Vite, PWA, Python (dados sintéticos), Vitest, pytest, GitHub Actions/Pages.
5. Como rodar localmente: pré-requisitos e comandos (npm install, python scripts/gerar\_dados.py, npm run dev, npm test).
6. Estrutura de pastas.
7. Roadmap: as 5 sprints com status (✅/🚧) e link para a release de cada uma.
8. Privacidade e LGPD: dados 100% sintéticos, campos mascarados, o que o sistema real precisaria (servidor, autenticação, logs em banco).
9. Transparência sobre IA: tabela Parte do sistema | Gerado por IA? | Revisado por | Prompt principal usado.
10. Equipe: [NOMES E PAPÉIS DO GRUPO].

### PRINT DA TELA NO README

Salvem o print em docs/img/painel.png e usem ![Painel](docs/img/painel.png). Conferiram que o print não mostra nenhum dado que pareça real?

## PARTE 5

# Biblioteca de prompts

Prompts prontos para consultar quando precisar: planejar, construir cada camada, depurar erros e revisar o código.

1. Prompts de planejamento e entendimento
2. Prompts de construção por camada
3. Prompts de depuração: quando algo dá errado
4. Prompts de revisão, refatoração e documentação
5. Prompts ruins × prompts bons

§ 26

## Prompts de planejamento e entendimento

Para usar no início de cada sprint ou quando o grupo estiver travado.

### P1 — Planejar a sprint PAINEL AGENT DO ANTIGRAVITY

PROMPT

Leia @docs/sprint.md e @docs/REGRAS-DO-PROJETO.md. Estamos começando a Sprint [N].  
Liste as tarefas desta sprint na ordem em que devem ser feitas (dependências primeiro),  
estime cada uma em P/M/G e aponte quais podem ser feitas em paralelo por pessoas diferentes do grupo.

### P2 — Entender um arquivo PAINEL AGENT DO ANTIGRAVITY

PROMPT

Explique, como para alguém que nunca programou, o que faz o arquivo @ [ARQUIVO].  
Use no máximo 10 linhas, e depois liste as 3 partes mais importantes do código com o número da linha.

### P3 — Diagrama da arquitetura PAINEL AGENT DO ANTIGRAVITY

PROMPT

Desenhe em Mermaid (flowchart LR) a arquitetura atual do projeto: pastas src/, scripts/, public/data/,  
o fluxo de dados do Python até a tela e o deploy pelo GitHub Actions. Salve em docs/arquitetura.md.

### P4 — Onde estamos? PAINEL AGENT DO ANTIGRAVITY

PROMPT

Compare o que já foi implementado com @docs/sprint.md. Monte uma tabela Tarefa | Status (feito/parcial/não iniciado) |  
Evidência (arquivo ou teste) | O que falta. Seja rigoroso: sem teste, é "parcial".

## P5 — Cortar escopo com critério

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Tenho [N HORAS] de aula para terminar o MVP. Olhando @docs/sprint.md e o que já existe, proponha o corte de escopo:  
o que é essencial para a demonstração, o que pode ser simplificado e o que fica para depois. Justifique cada item.

§ 27

## Prompts de construção por camada

### Dados (Python)

#### C1 — Nova entidade relacionada

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Acrescente ao scripts/gerar\_dados.py a entidade [ENTIDADE] com os campos de @src/domain/tipos.ts, ligada a [OUTRA ENTIDADE] pelo campo [CHAVE]. Toda chave estrangeira deve existir (sem órfãos). Adicione um teste pytest que prova isso.

#### C2 — Dados com história para contar

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Os dados estão "bonitos demais". Ajuste o gerador para ter padrões realistas: sazonalidade (mais pedidos em [MESES]), [UNIDADE] com o dobro de devoluções, e 3% de registros com campo opcional vazio.  
Documente esses padrões em docs/dados.md para usarmos no pitch.

### Regras e serviços (TypeScript)

#### C3 — Regra nova, testes primeiro

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Implemente a regra [RNOX: DESCRIÇÃO] como função pura em src/domain/regras.ts.  
Antes do código, escreva os testes Vitest (incluindo casos de limite) e me mostre. Depois implemente até passarem.

#### C4 — Cálculo de KPIs

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Crie a função calcularKpis(pedidos) em src/domain/kpis.ts que devolve [LISTA DE KPIs DO CONTEXTO.MD], cada um com valor, meta, variação percentual e status ('ok' | 'atencao' | 'critico'). Com testes.

### Telas (TypeScript + CSS)

#### C5 — Componente reutilizável

PAINEL AGENT DO ANTIGRAVITY

PROMPT

Crie o componente src/components/kpiCard.ts: recebe {titulo, valor, meta, status} e devolve o HTML do cartão.  
Cor pelo status (verde/âmbar/vermelho) com texto além da cor (acessibilidade). Use em src/pages/painel.ts.

## 💬 C6 — Exportar CSV PAINEL AGENT DO ANTIGRAVITY

PROMPT

Adicione em `src/pages/[TELA].ts` um botão "Exportar CSV" que baixa os dados filtrados na tela, separador `;`, UTF-8 com BOM (abre certo no Excel), nome do arquivo com a data. Registre a exportação no log de auditoria.

## 💬 C7 — Versão celular PAINEL AGENT DO ANTIGRAVITY

PROMPT

Deixe a tela `[TELA]` responsiva: no celular (até 480px) a tabela vira lista de cartões e os filtros ficam num painel recolhível. Altere somente o CSS e o mínimo de HTML necessário.

## 💬 C8 — Tema escuro PAINEL AGENT DO ANTIGRAVITY

PROMPT

Adicione um modo escuro ao app: variáveis CSS para as cores, respeitando `prefers-color-scheme`, com botão para alternar e a escolha salva no `localStorage`. Garanta contraste AA nos dois temas.

## PWA

## 💬 C9 — Aviso de offline PAINEL AGENT DO ANTIGRAVITY

PROMPT

Mostre um aviso "Você está offline – exibindo os últimos dados salvos" quando `navigator.onLine` for falso, e esconda quando a conexão voltar. Não bloqueie o uso do app.

## § 28

## Prompts de depuração: quando algo dá errado

O segredo é dar ao agente o **erro completo**, o que vocês fizeram e o que esperavam. "Não funciona" não é um relato de erro.

## 💬 D1 — Erro no terminal PAINEL AGENT DO ANTIGRAVITY

PROMPT

Rodei `[COMANDO, ex.: npm run build]` e deu o erro abaixo. Eu esperava `[RESULTADO ESPERADO]`. Explique a causa em linguagem simples, mostre o arquivo e a linha, e proponha a MENOR correção possível. Não altere outros arquivos.  
Erro completo:  
`[COLE O ERRO INTEIRO DO TERMINAL]`

## 💬 D2 — Tela em branco PAINEL AGENT DO ANTIGRAVITY

PROMPT

A tela `[TELA]` abre em branco. No console do navegador (F12 → Console) aparece:  
`[COLE AS MENSAGENS VERMELHAS]`  
Investigue a causa antes de mudar código: me diga as hipóteses em ordem de probabilidade e como confirmar cada uma.

### D3 — Funciona local, quebra no Pages PAINEL AGENT DO ANTIGRAVITY

PROMPT

Localmente funciona, mas no GitHub Pages ([https://\[USUARIO\].github.io/\[REPO\]/](https://[USUARIO].github.io/[REPO]/)) **[O QUE ACONTECE]**.  
Verifique: base do vite.config.ts, caminhos com "/" fixo, uso de import.meta.env.BASE\_URL, rotas sem hash, start\_url/scope do manifesto e maiúsculas no nome do repositório.

### D4 — Teste falhando PAINEL AGENT DO ANTIGRAVITY

PROMPT

O teste **[NOME DO TESTE]** está falhando. Antes de mexer: me diga se o erro está no TESTE ou no CÓDIGO, comparando com a regra em @docs/contexto.md. Não altere o teste só para ele passar.

### D5 — Agente em loop PAINEL AGENT DO ANTIGRAVITY

PROMPT

Você já tentou corrigir isto 3 vezes sem sucesso. Pare. Resuma o que tentou, por que cada tentativa falhou, e proponha uma abordagem diferente e mais simples. Espere meu OK.

### D6 — PWA preso na versão antiga PAINEL AGENT DO ANTIGRAVITY

PROMPT

Depois de publicar uma versão nova, o app instalado no celular continua mostrando a versão antiga. Verifique a configuração do vite-plugin-pwa (registerType, aviso de atualização) e me explique como forçar a atualização.

---

§ 29

## Prompts de revisão, refatoração e documentação

### R1 — Refatorar com segurança PAINEL AGENT DO ANTIGRAVITY

PROMPT

Revise @ **[ARQUIVO]** procurando código duplicado, funções com mais de 40 linhas e nomes pouco claros. Proponha a refatoração em passos pequenos, cada um mantendo os testes passando. Não mude comportamento.

### R2 — Cobrir o que falta PAINEL AGENT DO ANTIGRAVITY

PROMPT

Rode `npm test` e `python -m pytest -q scripts` e me diga quais funções de `src/domain` e `scripts` **NÃO** têm teste. Escreva os testes que faltam para as 3 mais importantes para o negócio.

### R3 — Acessibilidade PAINEL AGENT DO ANTIGRAVITY

PROMPT

Audite a acessibilidade das telas: labels, ordem de tabulação, foco visível, contraste, textos alternativos, uso de cor como única informação. Tabela Problema | Tela | Correção. Depois corrija os 5 mais graves.

Rodei o Lighthouse (F12 → Lighthouse) e o resultado foi: **[COLE AS NOTAS E OS ITENS APONTADOS]**.  
Explique cada item em linguagem simples e corrija os que mais tiram ponto, um por vez, sem mudar o visual.

Gere o roteiro do pitch de 3 minutos do MVP: Minuto 1 — o problema com número; Minuto 2 — a demonstração (qual tela mostrar, em qual papel, qual clique); Minuto 3 — a recomendação e o próximo passo para a SESP. Use os dados de public/data e os KPIs de @docs/contexto.md.

§ 30

## Prompts ruins × prompts bons

Os mesmos pedidos, antes e depois. A diferença está no contexto, no escopo e no critério de pronto.

### ✗ PROMPT VAGO

Faz o sistema.

### ✓ PROMPT TÉCNICO

Leia @docs/sprint.md e execute só a tarefa S1.2 (gerador de dados). Mostre o plano antes. Pronto quando o pytest passar.

### ✗ PROMPT VAGO

Arruma o erro.

### ✓ PROMPT TÉCNICO

npm run build falhou com o erro abaixo. Explique a causa e proponha a menor correção, sem mexer em outros arquivos. [erro completo]

### ✗ PROMPT VAGO

Deixa o painel bonito.

### ✓ PROMPT TÉCNICO

No painel, os KPIs estão sem hierarquia. Aumente o valor para 32px, rótulo em cinza 13px, cor pelo status. Só src/style.css.

### ✗ PROMPT VAGO

Coloca login no sistema.

### ✓ PROMPT TÉCNICO

Implemente o seletor 'Ver como' com os 4 papéis de @docs/contexto.md, usando pode() de src/auth/papeis.ts. Sem senha: é simulação.

**X PROMPT VAGO**

Usa os dados da planilha da secretaria.

**✓ PROMPT TÉCNICO**

Gere dados sintéticos com a mesma estrutura de colunas abaixo, sem nenhum valor real.  
[só os nomes das colunas]

**X PROMPT VAGO**

Melhora o código.

**✓ PROMPT TÉCNICO**

Revise @src/pages/pedidos.ts: aponte duplicações e cálculos que deveriam estar em src/domain. Tabela de achados, sem alterar nada.

## Fechamento

Os problemas mais comuns e como resolver, o checklist de entrega, o glossário e um quiz para revisar.

1. Problemas comuns e como resolver
2. Checklist de entrega do Dia 8 (Definição de Pronto)
3. Glossário rápido
4. Quiz rápido

§ 31

### Problemas comuns e como resolver

| Sintoma                                                              | Causa provável                                   | Solução                                                                                                              |
|----------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Site publicado em branco                                             | base do Vite diferente do nome do repositório    | Corrigir vite.config.ts (maiúsculas importam), commit e push                                                         |
| 404 ao carregar /data/*.json                                         | Caminho com "/" fixo                             | Usar import.meta.env.BASE_URL                                                                                        |
| 404 ao recarregar numa tela                                          | Rota sem hash (/pedidos)                         | Usar rotas por hash (#/pedidos)                                                                                      |
| Actions: erro no job deploy dizendo que o Pages não está configurado | Pages não configurado para Actions               | Settings → Pages → Source: <b>GitHub Actions</b>                                                                     |
| Actions: npm ci falhou                                               | package-lock.json não commitado ou desatualizado | Rodar npm install e commitar o lock                                                                                  |
| Actions: pip install falhou                                          | Falta o requirements.txt                         | pip freeze > requirements.txt e commitar                                                                             |
| "npm.ps1 não pode ser carregado"                                     | Política de scripts do PowerShell                | Set-ExecutionPolicy -Scope CurrentUser RemoteSigned                                                                  |
| python abre a Microsoft Store                                        | Alias do Windows                                 | Desligar os aliases ou usar py                                                                                       |
| Push rejeitado ( <i>rejected, fetch first</i> )                      | Um colega enviou antes                           | <b>Pull</b> (Sync) primeiro, depois push                                                                             |
| Conflito de merge                                                    | Duas pessoas editaram a mesma linha              | Abrir o arquivo e escolher <i>Accept Current/Incoming/Both</i> ; na dúvida, pedir ao agente para explicar o conflito |
| App não atualiza após o deploy                                       | Service worker com versão antiga                 | F12 → Application → Unregister + Ctrl+Shift+R                                                                        |
| node_modules apareceu no GitHub                                      | .gitignore ausente no primeiro commit            | git rm -r --cached node_modules (tira do Git sem apagar do disco) e commit                                           |
| O agente mexeu em arquivos demais                                    | Prompt sem limite de escopo                      | Source Control → Discard Changes nos arquivos indevidos; repetir com "altere somente X"                              |

## Checklist de entrega do Dia 8 (Definição de Pronto)

### Repositório

- ☐ Repositório público no GitHub com todos os integrantes como colaboradores
- ☐ `docs/contexto.md`, `docs/sprint.md` e `docs/REGRAS-D0-PROJETO.md` commitados
- ☐ `.gitignore` funcionando: sem `node_modules`, `dist` e `.venv` no repositório
- ☐ Commits pequenos, com mensagens no padrão Conventional Commits

### Código

- ☐ `scripts/gerar_dados.py` gera os JSON sintéticos; `pytest` passa (LGPD garantida)
- ☐ Regras de negócio em `src/domain` com testes; `npm test` passa
- ☐ Seletor de papel funcionando; cada papel vê só o que pode
- ☐ Painel com os KPIs do contexto.md, passando no teste dos 5 segundos
- ☐ PWA: instalável, com ícone e abrindo offline

### Publicação

- ☐ Workflow do GitHub Actions verde na aba Actions
- ☐ Site abrindo em `https://SEU-USUARIO.github.io/NOME-D0-REPO/`, inclusive no celular
- ☐ README com link, como rodar, roadmap, LGPD e transparência sobre IA
- ☐ Release/tag da última sprint concluída
- ☐ Pitch de 3 minutos ensaiado com o site publicado

## Glossário rápido

### Construção

#### Vite

Ferramenta que roda o projeto em desenvolvimento ( `npm run dev` ) e gera a versão final ( `npm run build` ).

#### TypeScript (TS)

JavaScript com tipos. Aponta erros antes de rodar. Vira JS no build.

#### Build / dist/

O processo de compilar o projeto e a pasta com o resultado, que é o que vai para o Pages.

#### npm / package.json

Gerenciador de pacotes do Node e o arquivo que lista dependências e scripts.

#### venv / pip

Ambiente isolado do Python e o instalador de pacotes Python.

#### Vitest / pytest

Ferramentas de teste automatizado para TypeScript e para Python.

#### Função pura

Função que só depende das entradas e não mexe em tela nem rede. Fácil de testar.

### Publicação

#### PWA

Site que se instala como app, com ícone, tela cheia e modo offline.

#### Service worker

JavaScript que roda em segundo plano e guarda arquivos para o modo offline.

#### Manifesto

Arquivo com nome, ícones e cores do app instalado.

#### GitHub Pages

Hospedagem gratuita de sites estáticos a partir de um repositório.

#### GitHub Actions / workflow

Automação do GitHub; o workflow é o arquivo YAML com os passos.

#### base

Subpasta onde o site fica publicado ( `/nome-do-repo/` ).

#### Rota por hash

Endereço de tela depois do # ( `#/painel` ); funciona em hospedagem estática.

#### Pull Request / Tag

Pedido de revisão para juntar uma branch na main; marco de versão ( `v1.0.0` ).

## Quiz rápido

O site publicado abre em branco, mas no `npm run dev` funciona. Qual a primeira coisa a conferir?

- a. O token do GitHub
- b. O `base` do `vite.config.ts`
- c. A versão do Python
- d. O tema escuro

**RESPOSTA (b)** O `base` do `vite.config.ts`. O Pages publica em `/nome-do-repo/`; sem o base certo, o navegador procura os arquivos na raiz.

Onde deve ficar o mascaramento do CPF?

- a. No CSS da tela
- b. No script Python, antes de gerar o JSON
- c. No README
- d. No service worker

**RESPOSTA (b)** No script Python, antes de gerar o JSON. Se o dado completo chega ao JSON, ele é público no GitHub Pages, mesmo que a tela esconda.

Por que usar rotas como `#/pedidos` em vez de `/pedidos` ?

- a. É mais bonito
- b. O GitHub Pages não conhece as rotas do app e daria 404 ao recarregar
- c. O TypeScript exige
- d. Para o PWA ter ícone

**RESPOSTA (b)** O GitHub Pages não conhece as rotas do app e daria 404 ao recarregar.

O agente quer rodar `npm create vite . --overwrite` na pasta com o contexto.md. O que fazer?

- a. Aprovar, é rápido
- b. Recusar: `--overwrite` apaga os arquivos existentes
- c. Aprovar e recuperar depois pelo Pages
- d. Rodar como administrador

**RESPOSTA (b)** Recusar: `--overwrite` apaga os arquivos existentes.

**Qual é o tamanho ideal de um pedido ao agente?**

- a. A sprint inteira
- b. Uma tarefa do sprint.md, com critério de pronto
- c. O projeto todo de uma vez
- d. Uma linha de código

**RESPOSTA** (b) Uma tarefa do sprint.md, com critério de pronto.

**Um teste falhou. O agente propõe mudar o teste para ele passar. Qual a atitude correta?**

- a. Aceitar
- b. Verificar no contexto.md se o erro está no teste ou no código
- c. Apagar o teste
- d. Desligar o Vitest no workflow

**RESPOSTA** (b) Verificar no contexto.md se o erro está no teste ou no código.